Lundi 18 septembre 2023
Fondements programmation
CM1

But : Langage
Outils pour garantir que notre langage se comporte comme il doit se comporter

let x = 1 in x+2 ✓ (valide)

let 2x3 = y in y X (non valide)

let g ::= Num == 2+2 in y + Unit (Unit fait pas partie des expressions donc X)

let y ::= Num → (Num → Unit) = 22 in y+3

let (x : T) = e1 in e2

(Point de départ →) Expression

(Peut se réécrire dans un let) let Var(x) = Expression in Expression

Expression : Num(3)

Expression : Expression + Expression

Expression : Var(x)

Expression : Num(12)

(expression NT donc se réécrivent en autre chose)

(c'est comme ça que le compilateur sait si le langage est valide ou non)

lexeur : découper en briques de base

let	x	=	y	+	3	in	x	+	12

parseur : vérifier si une expression qu'on a le droit d'écrire.

---

let y = (let x = 3 in x) in y+5

Expression : let var(y) = Expression in Expression

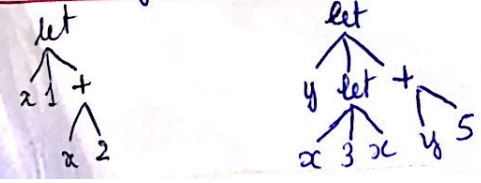
let Var(x) = Expression in Expression

Num(3) Var(x)

Expr + Expr

Var(y) Num(5)

Abstract Syntax Tree : manière plus condensée d'écrire une Expression (pas de pt de départ)



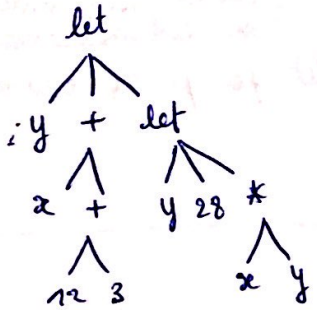
Scanned with CamScanner

Notre exemple :

let y = x+12+3 in let y = 28 in x*y

(ambigu → parenthèses)

(on a été associatif à droite ici naturellement)



Donc changer grammaire et indiquer priorité

!b == c
(!b) == c
!(b == c)

⚠ Les 2 sont valides mais celui qu'on retient est celui qui est associatif à droite.

① if b then (if c then), else ✗
② if b then (if c then, else) ✓

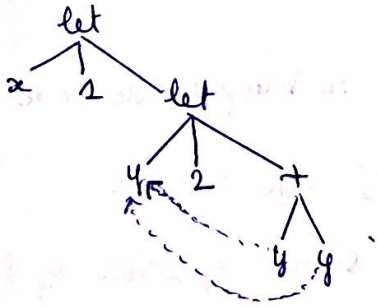
(ternaire prioritaire sur le binaire)

① (true ? false : true) ? true : true ✗
② true ? false : (true ? true : true) ✓

(associatif à droite)

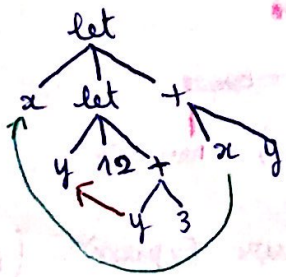
let x = 1 in let y = 2 in y+y

↓
sert à rien
variables liées x et y



let x = (let y = 12 in y+3) in x+y

x = 15 15+y



Scanned with CamScanner